Lab 7: Fuzzing for Input Validation Bugs

Name: Rishi Jain
SRN: PES1UG24CS375
Project: L6_L7_11 - Amusement Park Ride Access
Tool: Python 3.12.5, Hypothesis 6.170.15, pytest 8.3.5

Step 2: Running the buggy application

The original `processor.py` crashes on malformed input. Input `FastPass` (no colon) raises a **ValueError**, and `10/0` raises a **ZeroDivisionError**.

```
PS C:\Users\rishi\lab7_fresh\Lab 7> cd C:\Users\rishi\lab7_fresh\"Lab 7"
PS C:\Users\rishi\lab7_fresh\Lab 7> git restore processor.py
PS C:\Users\rishi\lab7_fresh\Lab 7> python processor.py
==== Running validate_height_and_fastpass ====
Enter rider spec (Height:FastPass, e.g. 140:YES): FastPass
Traceback (most recent call last):
  File "C:\Users\rishi\lab7_fresh\Lab 7\processor.py", line 41, in <module>
    main()
  File "C:\Users\rishi\lab7_fresh\Lab 7\processor.py", line 30, in main
    print(validate_height_and_fastpass(r_input))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\rishi\lab7_fresh\Lab 7\processor.py", line 2, in validate_height_and_fastpass
    height_str, pass_str = rider_spec.strip().split(':')
    ^^^^^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
PS C:\Users\rishi\lab7_fresh\Lab 7>
```

Figure 1: ValueError in validate_height_and_fastpass

```
PS C:\Users\rishi\lab7_fresh\Lab 7> python processor.py
==== Running validate_height_and_fastpass ====
Enter rider spec (Height:FastPass, e.g. 140:YES): 140:YES
Height 140cm: Approved (FastPass Priority)

==== Running compute_dispatch_throughput ====
Enter riders/mins (e.g. 48/2.5): 10/0
Traceback (most recent call last):
  File "C:\Users\rishi\lab7_fresh\Lab 7\processor.py", line 41, in <module>
    main()
  File "C:\Users\rishi\lab7_fresh\Lab 7\processor.py", line 34, in main
    print("Throughput (riders/min):", compute_dispatch_throughput(t_input))
                                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\rishi\lab7_fresh\Lab 7\processor.py", line 14, in compute_dispatch_throughput
    rate = riders / cycle_mins
           ~~~~~~~^~~~~~~~~~~~
ZeroDivisionError: float division by zero
PS C:\Users\rishi\lab7_fresh\Lab 7>
```

Figure 2: ZeroDivisionError in compute_dispatch_throughput

Step 4: Hypothesis tests against the buggy code

`pytest test_processor.py` run on the buggy script: **11 failed, 4 passed**. Hypothesis found crashes on empty strings, `None`, bytes, a missing separator, division by zero, `nan`/`inf` values, out-of-range heights and unknown zone codes.

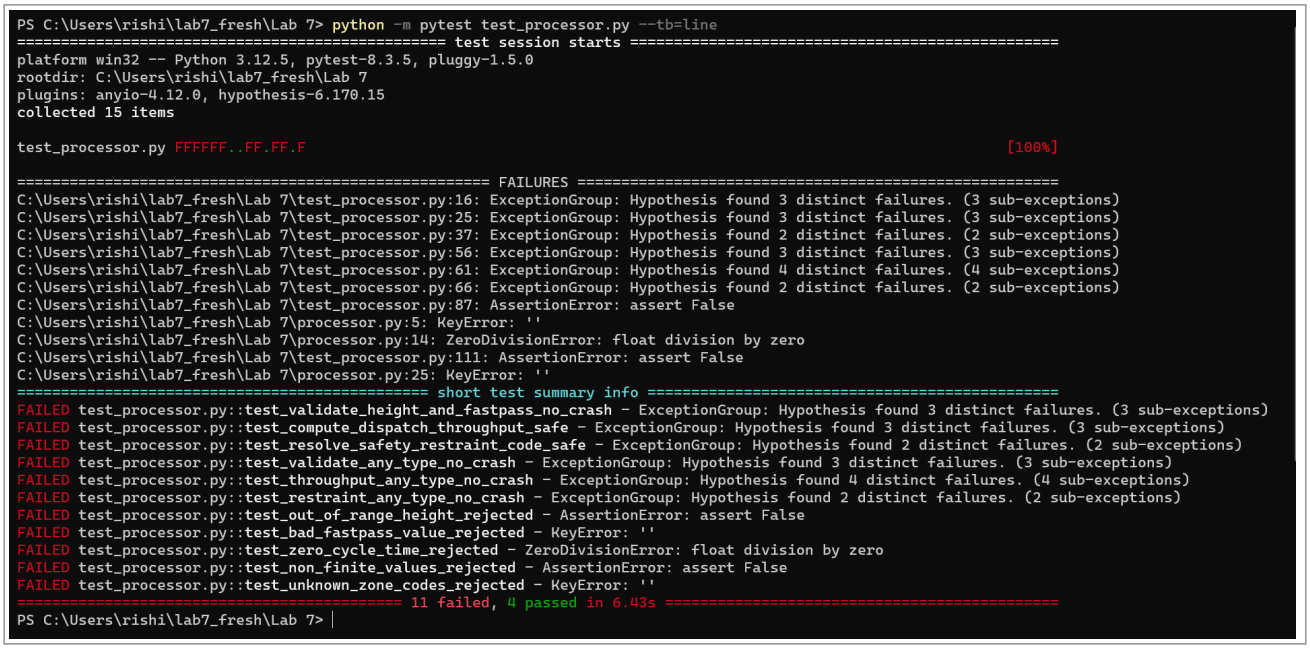


Figure 3: pytest output on the buggy processor.py (11 failed, 4 passed)

Bugs found and how they were fixed

Function	Bug found by fuzzing	Fix applied
validate_height_and_fastpass	No ':' gives ValueError; None/bytes crash; non-numeric height; bad FastPass gives KeyError; height 0, negative or huge accepted	Type check, exactly 2 parts, try/except around int(), height range 1-300, FastPass checked against YES/NO
compute_dispatch_throughput	Division by zero; nan/inf/1e999 accepted; negative riders; missing or extra '/' crashes	math.isfinite checks, cycle time must be > 0, riders >= 0, exactly 2 parts, try/except around float()
resolve_safety_restraint_code	Empty string or unknown prefix gives KeyError; None crashes; 'CSTXYZ' wrongly matched CST via [:3]	Type check, split on '_' to get the prefix, return an error message for unknown prefixes

Step 5: Running the fixed application

The fixed `processor.py` validates every input and returns a clear *Invalid input* message instead of crashing. Valid input still gives the original output format.

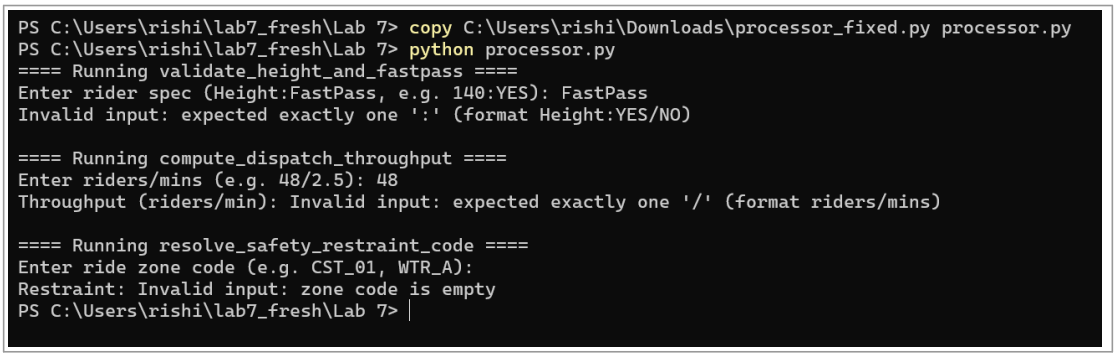


Figure 4: Malformed inputs (FastPass, 48, empty zone code) now handled gracefully

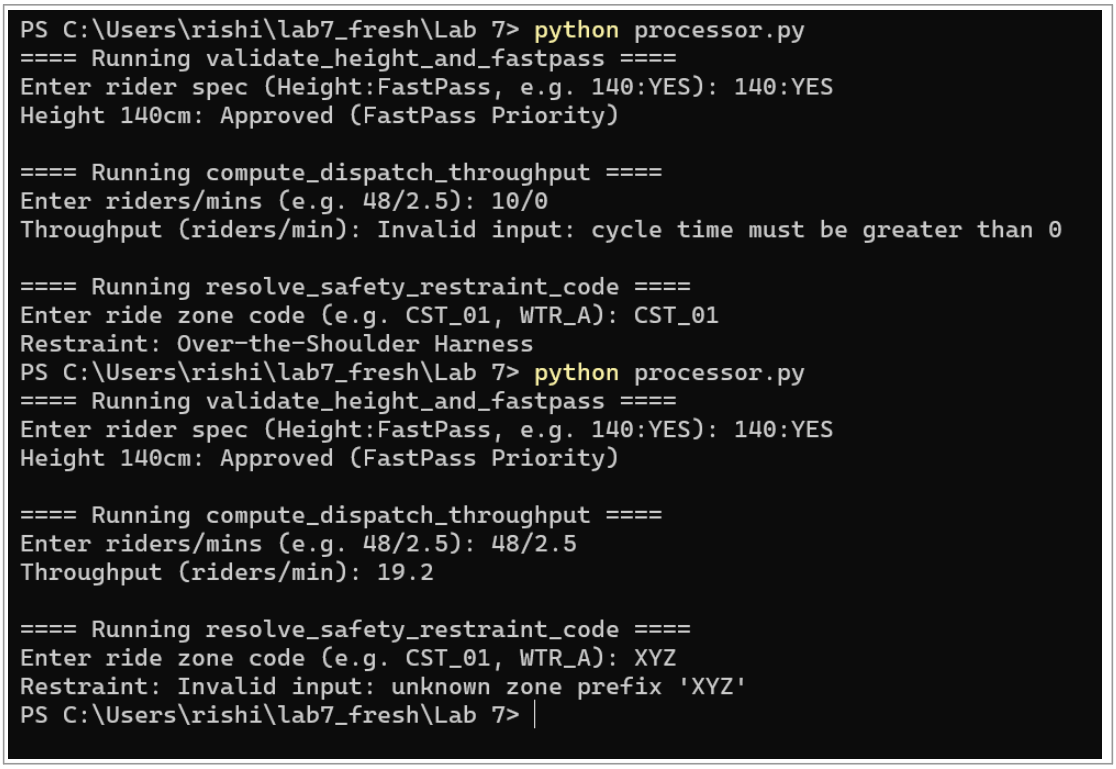


Figure 5: 10/0 and unknown zone code XYZ handled; valid input 48/2.5 gives 19.2

Step 6: Hypothesis tests against the fixed code

Re-running `pytest test_processor.py` on the fixed script: **all 15 tests pass**.

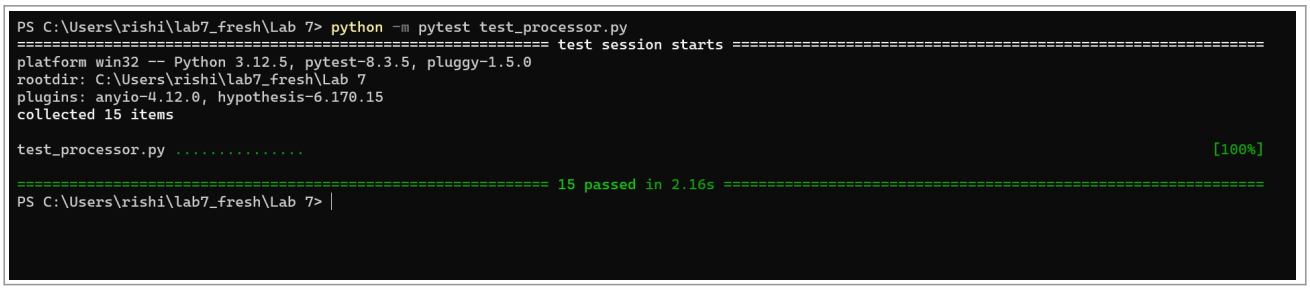


Figure 6: pytest output on the fixed processor.py (15 passed)

Source Code 1: processor.py (fixed)

All three functions now validate their input and return an error message instead of raising an exception.

```
import math

MIN_HEIGHT_CM = 120      # minimum height to be allowed on the ride
MAX_HEIGHT_CM = 300      # anything above this is treated as bad data


def validate_height_and_fastpass(rider_spec):
    """Input format 'Height:FastPass', e.g. '140:YES'.
    Returns a status string; never raises on bad input."""
    if not isinstance(rider_spec, str):
        return "Invalid input: rider spec must be text like 140:YES"

    parts = rider_spec.strip().split(':')
    if len(parts) != 2:
        return "Invalid input: expected exactly one ':' (format Height:YES/NO)"
    height_str, pass_str = parts

    try:
        height = int(height_str.strip())
    except (ValueError, OverflowError):
        return "Invalid input: height must be a whole number"

    if not (0 < height <= MAX_HEIGHT_CM):
        return f"Invalid input: height must be between 1 and {MAX_HEIGHT_CM} cm"

    pass_map = {"YES": True, "NO": False}
    key = pass_str.strip().upper()
    if key not in pass_map:
        return "Invalid input: FastPass must be YES or NO"
    has_fastpass = pass_map[key]

    eligible = height >= MIN_HEIGHT_CM
    status = "FastPass Priority" if has_fastpass else "Standard Entry"
    return f"Height {height}cm: {'Approved' if eligible else 'Denied'} ({status})"


def compute_dispatch_throughput(ride_metric_str):
    """Input format 'riders/mins', e.g. '48/2.5'.
    Returns riders per minute (float) or an error string; never raises."""
    if not isinstance(ride_metric_str, str):
        return "Invalid input: metric must be text like 48/2.5"

    parts = ride_metric_str.split('/')
    if len(parts) != 2:
        return "Invalid input: expected exactly one '/' (format riders/mins)"

    try:
        riders = float(parts[0].strip())
        cycle_mins = float(parts[1].strip())
    except (ValueError, OverflowError):
        return "Invalid input: riders and minutes must be numbers"

    # float() happily accepts 'nan', 'inf', '1e999' -> reject them
    if not (math.isfinite(riders) and math.isfinite(cycle_mins)):
        return "Invalid input: values must be finite numbers"
    if riders < 0:
        return "Invalid input: riders cannot be negative"
    if cycle_mins <= 0:
        return "Invalid input: cycle time must be greater than 0"  # avoids ZeroDivisionError

    try:
        rate = riders / cycle_mins
    except (ZeroDivisionError, OverflowError):
        return "Invalid input: could not compute throughput"
    if not math.isfinite(rate):
        return "Invalid input: throughput out of range"
    return round(rate, 2)


def resolve_safety_restraint_code(zone_code):
```

```
    """Zone code like 'CST_01' or 'WTR_A'. Returns restraint type or an error string."""
    if not isinstance(zone_code, str):
        return "Invalid input: zone code must be text like CST_01"

    code = zone_code.strip().upper()
    if not code:
        return "Invalid input: zone code is empty"

    prefix = code.split('_')[0]     # 'CST_01' -> 'CST'; rejects 'CSTXYZ'
    restraint_types = {
        "CST": "Over-the-Shoulder Harness",
        "WTR": "Lap Bar and Seatbelt",
        "DRP": "Hydraulic Shoulder Vest",
        "WHL": "Manual Safety Bar",
    }
    if prefix not in restraint_types:
        return f"Invalid input: unknown zone prefix '{prefix[:10]}'"
    return restraint_types[prefix]


def main():
    print("==== Running validate_height_and_fastpass ====")
    r_input = input("Enter rider spec (Height:FastPass, e.g. 140:YES): ")
    print(validate_height_and_fastpass(r_input))

    print("\n==== Running compute_dispatch_throughput ====")
    t_input = input("Enter riders/mins (e.g. 48/2.5): ")
    print("Throughput (riders/min):", compute_dispatch_throughput(t_input))

    print("\n==== Running resolve_safety_restraint_code ====")
    z_input = input("Enter ride zone code (e.g. CST_01, WTR_A): ")
    print("Restraint:", resolve_safety_restraint_code(z_input))


if __name__ == "__main__":
    main()
```

Source Code 2: test_processor.py (Hypothesis fuzz tests)

Contains the three required no-crash tests (text and None), three any-type fuzz tests, and nine property tests on valid and invalid structured input.

```
import math

from hypothesis import given, settings, strategies as st
from processor import (
    validate_height_and_fastpass,
    compute_dispatch_throughput,
    resolve_safety_restraint_code,
)


# ---------------------------------------------------------------------------
# Part 1: "No crash" fuzz tests (text + None, the To-Do functions)
# ---------------------------------------------------------------------------

@given(st.text() | st.none())
def test_validate_height_and_fastpass_no_crash(s):
    try:
        result = validate_height_and_fastpass(s)
        assert isinstance(result, str)
    except Exception as e:
        assert False, f"validate_height_and_fastpass crashed with: {type(e).__name__}: {e}"


@given(st.text() | st.none())
def test_compute_dispatch_throughput_safe(s):
    try:
        result = compute_dispatch_throughput(s)
        # valid input -> finite float, invalid input -> error string
        assert isinstance(result, (float, str))
        if isinstance(result, float):
            assert math.isfinite(result)
    except Exception as e:
        assert False, f"compute_dispatch_throughput crashed with: {type(e).__name__}: {e}"


@given(st.text() | st.none())
def test_resolve_safety_restraint_code_safe(s):
    try:
        result = resolve_safety_restraint_code(s)
        assert isinstance(result, str)
    except Exception as e:
        assert False, f"resolve_safety_restraint_code crashed with: {type(e).__name__}: {e}"


# ---------------------------------------------------------------------------
# Part 2: Extra fuzzing - other input types (ints, lists, bytes, floats ...)
# ---------------------------------------------------------------------------

anything = st.one_of(
    st.none(), st.integers(), st.floats(), st.booleans(),
    st.binary(), st.lists(st.integers()), st.text(),
)


@given(anything)
def test_validate_any_type_no_crash(x):
    assert isinstance(validate_height_and_fastpass(x), str)


@given(anything)
def test_throughput_any_type_no_crash(x):
    assert isinstance(compute_dispatch_throughput(x), (float, str))


@given(anything)
def test_restraint_any_type_no_crash(x):
    assert isinstance(resolve_safety_restraint_code(x), str)


# ---------------------------------------------------------------------------
```

```
# Part 3: Property tests on VALID / structured inputs (correct behaviour)
# ---------------------------------------------------------------------------

@given(st.integers(min_value=120, max_value=250), st.sampled_from(["YES", "NO", "yes", " No "]))
def test_valid_tall_rider_is_approved(h, p):
    out = validate_height_and_fastpass(f"{h}:{p}")
    assert out.startswith(f"Height {h}cm: Approved")


@given(st.integers(min_value=1, max_value=119), st.sampled_from(["YES", "NO"]))
def test_short_rider_is_denied(h, p):
    assert validate_height_and_fastpass(f"{h}:{p}").startswith(f"Height {h}cm: Denied")


@given(st.integers(max_value=0) | st.integers(min_value=301))
def test_out_of_range_height_rejected(h):
    assert validate_height_and_fastpass(f"{h}:YES").startswith("Invalid")


@given(st.text(max_size=10).filter(lambda t: t.strip().upper() not in ("YES", "NO")))
def test_bad_fastpass_value_rejected(p):
    assert validate_height_and_fastpass(f"150:{p}").startswith("Invalid")


@given(st.integers(min_value=0, max_value=10_000),
       st.floats(min_value=0.1, max_value=1000, allow_nan=False, allow_infinity=False))
def test_valid_throughput_matches_formula(riders, mins):
    result = compute_dispatch_throughput(f"{riders}/{mins}")
    assert isinstance(result, float)
    assert result == round(riders / mins, 2)
    assert result >= 0


@given(st.integers(min_value=0, max_value=1000))
def test_zero_cycle_time_rejected(riders):
    assert isinstance(compute_dispatch_throughput(f"{riders}/0"), str)


@given(st.sampled_from(["nan", "inf", "-inf", "NaN", "Infinity", "1e999"]))
def test_non_finite_values_rejected(bad):
    assert isinstance(compute_dispatch_throughput(f"{bad}/2"), str)
    assert isinstance(compute_dispatch_throughput(f"10/{bad}"), str)


@given(st.sampled_from(["CST", "WTR", "DRP", "WHL"]),
       st.text(alphabet="ABCDEFGHIJ0123456789", max_size=5))
def test_known_zone_codes_resolve(prefix, suffix):
    code = prefix + ("_" + suffix if suffix else "")
    result = resolve_safety_restraint_code(code)
    assert not result.startswith("Invalid")
    # case-insensitivity + whitespace tolerance
    assert resolve_safety_restraint_code(f"  {code.lower()}  ") == result


@given(st.text(max_size=12).filter(lambda t: t.strip().upper().split("_")[0] not in ("CST", "WTR",
     "DRP", "WHL")))
def test_unknown_zone_codes_rejected(code):
    assert resolve_safety_restraint_code(code).startswith("Invalid")
```